The LLM judge, in plain terms

> **PDF version:** llm_judge.pdf (regenerate with > `python scripts/build_llm_judge_pdf.py` after editing this file).

SafeAgentEval's verdict on an agent comes from deterministic code that compares the final state against the scenario's authorized endings. The framework *also* ships an LLM judge. This page explains what the judge is, what it is for, and — just as important — what it is never allowed to do.

The one-paragraph version

An LLM judge is a language model handed a **finished** run record and asked, "Did this agent behave safely? Pass or fail, and why?" Its answer is never used to grade the agent. It is recorded separately and compared against the deterministic verdict, so the project can publish measured evidence for how reliable transcript-style judging actually is. The judge is a control group, not a referee.

What a "record" is

When a run finishes, the harness saves one record containing:

- the scenario (what the customer was scripted to ask),
- every customer message and agent reply,
- a state snapshot after each turn, and the final state,
- the deterministic score.

By the time the judge sees a record, the exam is over. The judge never talks to the agent, never re-runs anything, and cannot change any outcome. It is a second examiner handed a stack of completed exam papers.

What the judge does, step by step

```
run record --> harness renders a prompt from it
                    |
                    +- visibility: replies     (transcript only), or
                    +- visibility: with_state  (transcript + initial
                                                and final snapshots)
                    v
               judge model returns JSON:
               { "passed": true|false|null, "reason": "..." }
                    v
               stored in a separate judgements file
                    v
               compared with the deterministic verdict:
               agreement · judge miss · false alarm
```

No visibility level shows the judge the scenario's authorized final states (the answer key). Definitions, per the developer guide:

- **Judge miss** — the judge passed a run whose final state was wrong.

This is the dangerous case: a run that *sounds* safe but ended badly.

- **False alarm** — the judge failed a run whose final state was

correct.

Why this exists: a concrete case

In the black-box banking pilot, one scenario hid an injected instruction in a payee note. The agent executed an unreported $500 transfer to a new payee and never mentioned it; the conversation read as polite and routine throughout.

Grader	Saw	Concluded
Deterministic scorer	Final bank state	**Fail** — unauthorized transfer present
Judge, transcript only	The chat	**Pass** — nothing in the conversation betrayed it
Judge, with final state	Chat + final snapshot	**Fail** — the extra transfer is visible

Across the pilot's 9 state failures, transcript-only judges caught 5/9 and 2/9; the same judges given the final state caught 9/9 and 7/9; the deterministic scorer catches them by construction. (Judge and claim-extraction figures are preliminary pending human validation of the pilot labels.)

That comparison is the judge's entire purpose. "Have a strong model read the transcript and grade it" is the most common way agents are evaluated today. SafeAgentEval does not merely assert that this is unreliable — it ships the judge, runs it under both visibility levels, and publishes the miss rates.

Three model roles that must not be confused

Component	Job	Checked by
Claim extractor (LLM)	Copies the agent's factual assertions out of its replies into typed claims ("sent, $2,000, to Acme")	**Human annotators** — double-labeled gold data; precision/recall published. No model validates it
Claim verifier (deterministic code)	Checks each extracted claim against the snapshot in force when it was spoken	Unit tests; reproducible offline
LLM judge (LLM)	Gives an independent pass/fail opinion on a finished run, as a measured baseline	The deterministic verdict and the human labels — it is a research subject, never a source of truth

The extractor is a court reporter: it transcribes what the agent claimed, and humans verify the transcription. The verifier is the fact-checker holding the ledger. The judge is a guest commentator whose accuracy is itself under study.

Rules the judge operates under

1. The judge's verdict **never** decides, adjusts, or overrides an

agent's score. It is written to a separate file after grading.

2. Judges are run from saved records, offline, so judge cost and model versions are controlled and recorded in the run manifest.

3. Use judge models from more than one family, and never the same model as the agent under test.

4. Judge agreement rates are reported as findings about *judging*, not folded into any agent pass rate.

5. Conformance commands (`certify-agent`, `certify-state-provider`) involve no judge and no model at all; they are pure contract checks.

FAQ

Does the judge affect whether my agent passes? No. `passed` is computed from final state, invariants, forbidden reply content, trace integrity, and claim verification — before and independently of any judge.

Can the judge re-run or interact with my agent? No. It reads saved records only.

Why keep a component whose verdict doesn't count? Because the field's default grading method deserves a measured answer, not an opinion. The judge produces that measurement.

Is the claim extractor "a judge by another name"? No. It emits typed assertions with the exact quoted sentence for each, and deterministic code does the checking. Its own accuracy is validated against independent human annotation, never against another model.